

Introduction to AI Agents

Dr. Gabriel Jaffe

PEARC 2026

Scalable Computational Intelligence Group (SCI)

Texas Advanced Computing Center (TACC)

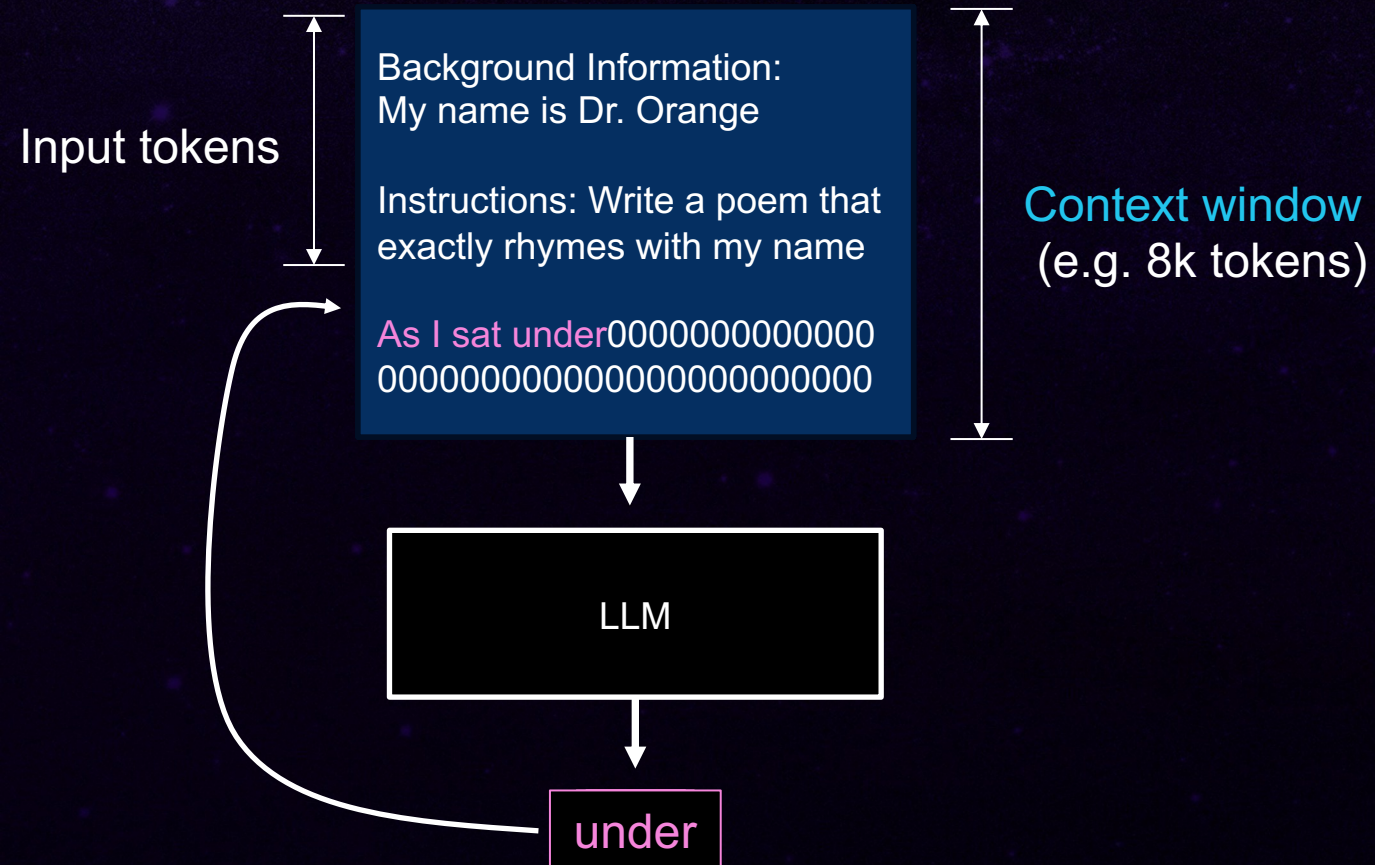


Outline

1. Background on LLMs
2. What even *is* an agent?
3. Examples of agentic systems

Using LLMs as the engine for an agent

Prompt



- LLMs can only make decisions based on
 1. Their general knowledge from training
 2. Information in the prompt
- Agentic software is dynamically building blocks of background information and instructions for an LLM prompt
- The LLM outputs are mapped to existing software tools that can be executed
 - e.g. "Call weather API to find current weather"

How fast can I run my model?

A little faster than you can read!

Model Family # of Parameters		iPhone 16 Pro Max tokens/s	Macbook Pro tokens/s	Vista GPU node tokens/s	Tejas tokens/s
Qwen3	1.7 billion	13	73	313	-
Llama3.1	8 billion	-	25	173	1001
Qwen3	30 billion	-	36	130	-
DeepSeek-R1	70 billion	-	-	36	-
DeepSeek-R1	671 billion	-	-	-	135

Time to generate 100,000 tokens for agentic workloads:

4 hours	40 minutes	10 minutes	2 minutes
---------	------------	------------	-----------

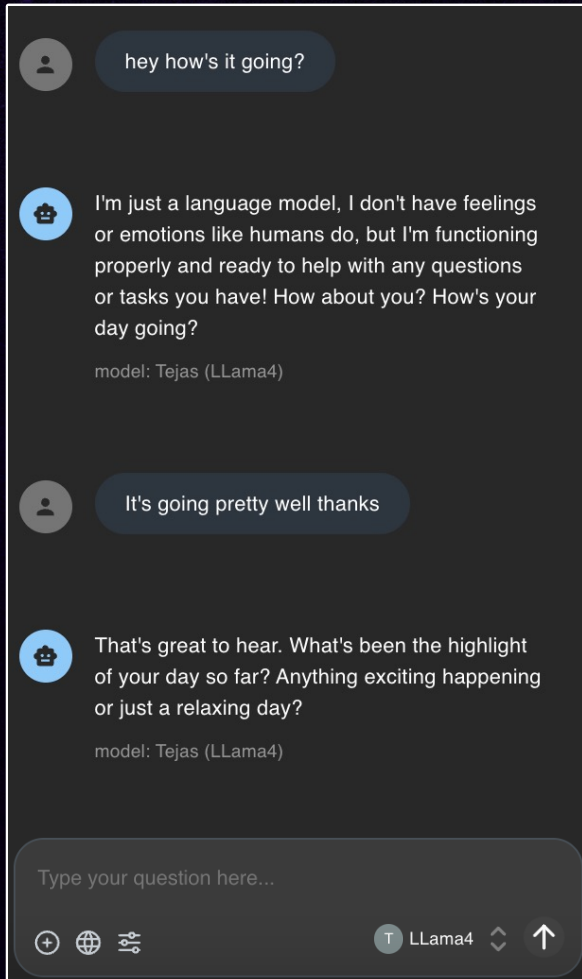
- Laptop is ~6x faster than phone
- 1 Vista GPU is ~4x faster than laptop
- Tejas is ~5x faster than 1 Vista GPU

Outline

1. Background on LLMs
2. *What even is an agent?*
3. Examples of agentic systems

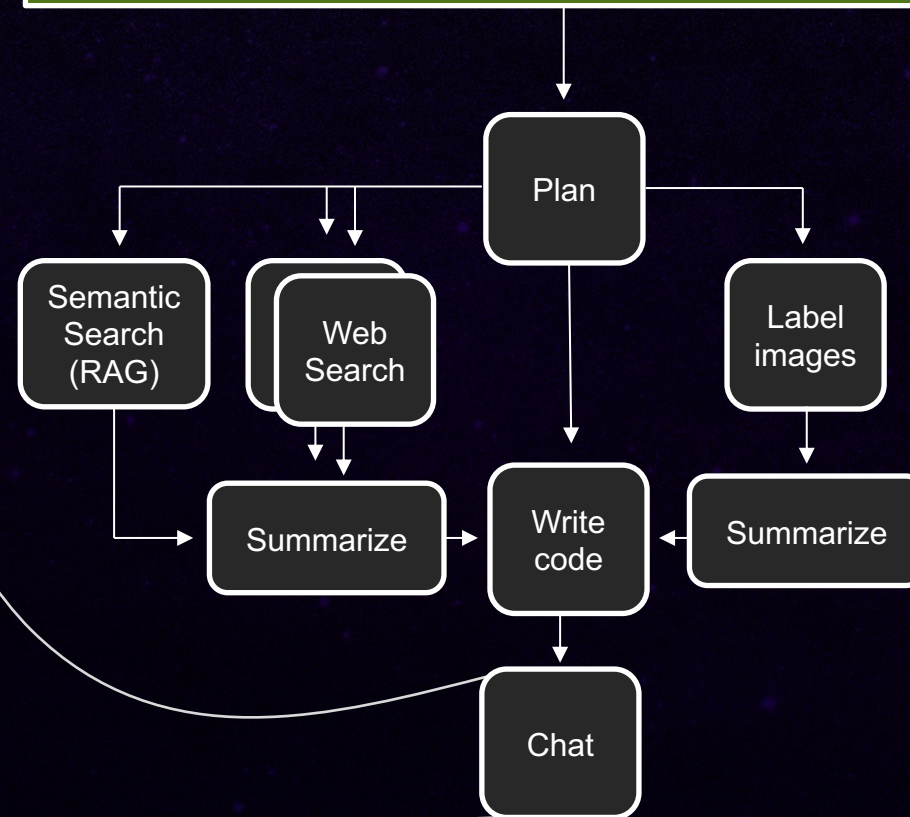
Chatbots are one module in agentic systems

Chatbot



Agent

User Input:
Write code to classify input images in the ~/data directory



Capabilities of LLM-based agentic systems

- LLMs can only directly provide solutions word problems
 - knowledge retrieval
 - structure unstructured text
 - generate code
 - classify text/images
 - solve logic puzzles (given X what should I do next?)
- Most of what agents *do* is autonomously execute existing software

How to build an agent checklist

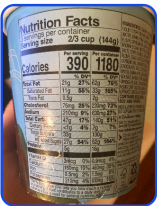
1. How do I organize the information that I send to the LLM so that it can solve my problem?
 - How much text can I input and generate at once?
 - How fast can I run my model?
2. What software do I need to write for my agent to use?

Example of agent-level tasks from the GAIA Benchmark

GAIA Benchmark Example Questions

Level 1

Question: What was the actual enrollment count of the clinical trial on *H. pylori* in acne vulgaris patients from Jan-May 2018 as listed on the NIH website?
Ground truth: 90



Level 2

Question: If this whole pint is made up of ice cream, how many percent above or below the US federal standards for butterfat content is it when using the standards as reported by Wikipedia in 2020? Answer as + or - a number rounded to one decimal place.
Ground truth: +4.6

Level 3

Question: In NASA's Astronomy Picture of the Day on 2006 January 21, two astronauts are visible, with one appearing much smaller than the other. As of August 2023, out of the astronauts in the NASA Astronaut Group that the smaller astronaut was a member of, which one spent the least time in space, and how many minutes did he spend in space, rounded to the nearest minute? Exclude any astronauts who did not spend any time in space. Give the last name of the astronaut, separated from the number of minutes by a semicolon. Use commas as thousands separators in the number of minutes.
Ground truth: White; 5876

Swarm Benchmark Performance

Table 1. Performance on the GAIA Benchmark (Mialon et al., 2023). Using our framework, we demonstrate significant improvements across several levels of difficulty. The ‘GPT-4 with plugins’ baseline is less significant since it involves the manual selection of the appropriate tools per question. We report the mean and standard deviation across 5 runs.

Method	Level 1	Level 2	Level 3	Avg.
GPT-3.5	7.55	4.65	0	4.85
GPT-4	15.09	2.33	0	6.06
GPT-4-Turbo	20.75	5.81	0	9.70
AutoGPT	13.21	0	3.85	4.85
GPTSwarm	30.56 $\pm$ 3.25	20.93 $\pm$ 1.27	3.85 $\pm$ 2.43	18.45
Improvement	47.3% $\uparrow$	260.2% $\uparrow$	0.0%	90.2% $\uparrow$
GPT4 with Plugins*	30.30	9.70	0	14.6

- GAIA benchmark¹ questions require reasoning, multi-modality processing, web browsing, and other tool use
- These are really hard tasks for LLMs, but easy for humans

[1] G. Mialon et al. “Gaia: a benchmark for general ai assistants” (2023) arXiv:2311.12983

Reinforcement learning (RL) versus LLM-based agents

AI Agent Definition: “An artificial entity capable of perceiving its surroundings using sensors, making decisions, and then taking actions in response using actuators.”

- Can be based around a reinforcement learning (RL) algorithm, or an LLM
- **RL-based agents** can struggle with complex tasks and their decision making can be inscrutable
 - e. g. in *long-horizon tasks* where the rewards during policy search are rare, the training times become extreme
- **LLM-based agents** utilize *goal decomposition* to break down goals into smaller subgoals
 - The LLM can act as a planner or scheduler that decides what tool it wants to use for a given subgoal
 - subgoals are accomplished using additional LLM calls or external tools via APIs

[Z. Xi et al. “The Rise and Potential of Large Language Model Based Agents: A Survey” *arXiv* \(2023\)](#)

Agents can perform planning and reflection on their memories as a form of learning

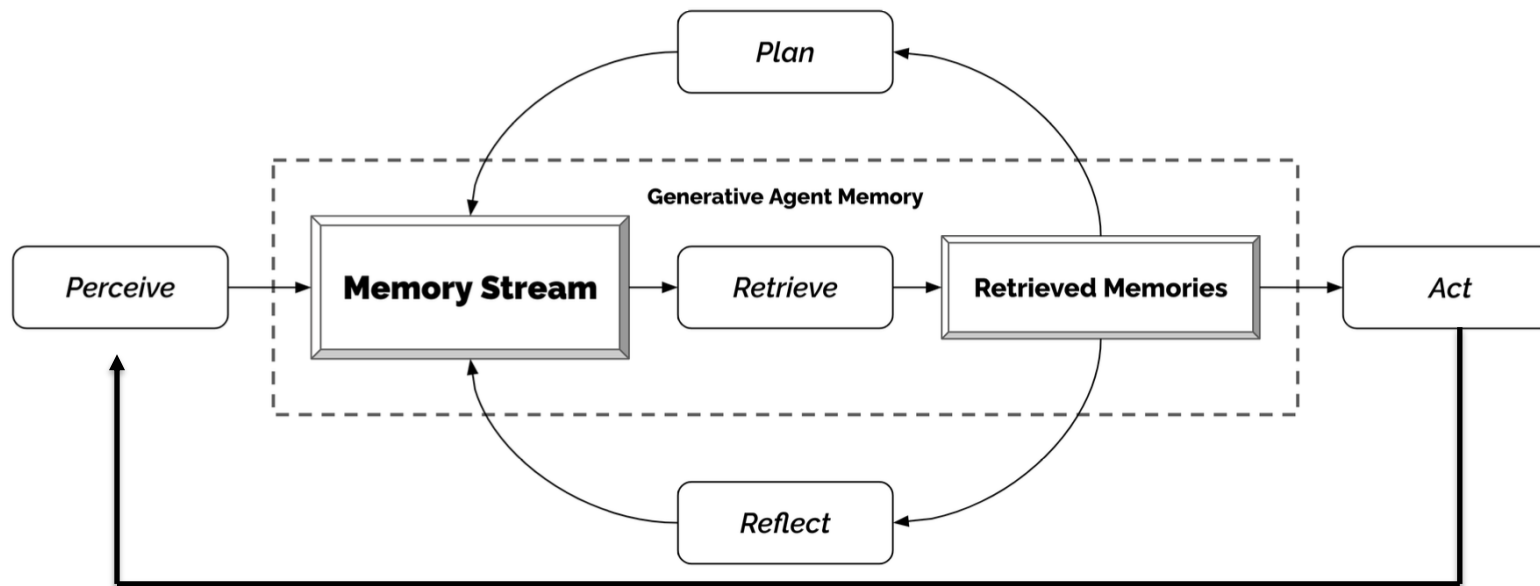
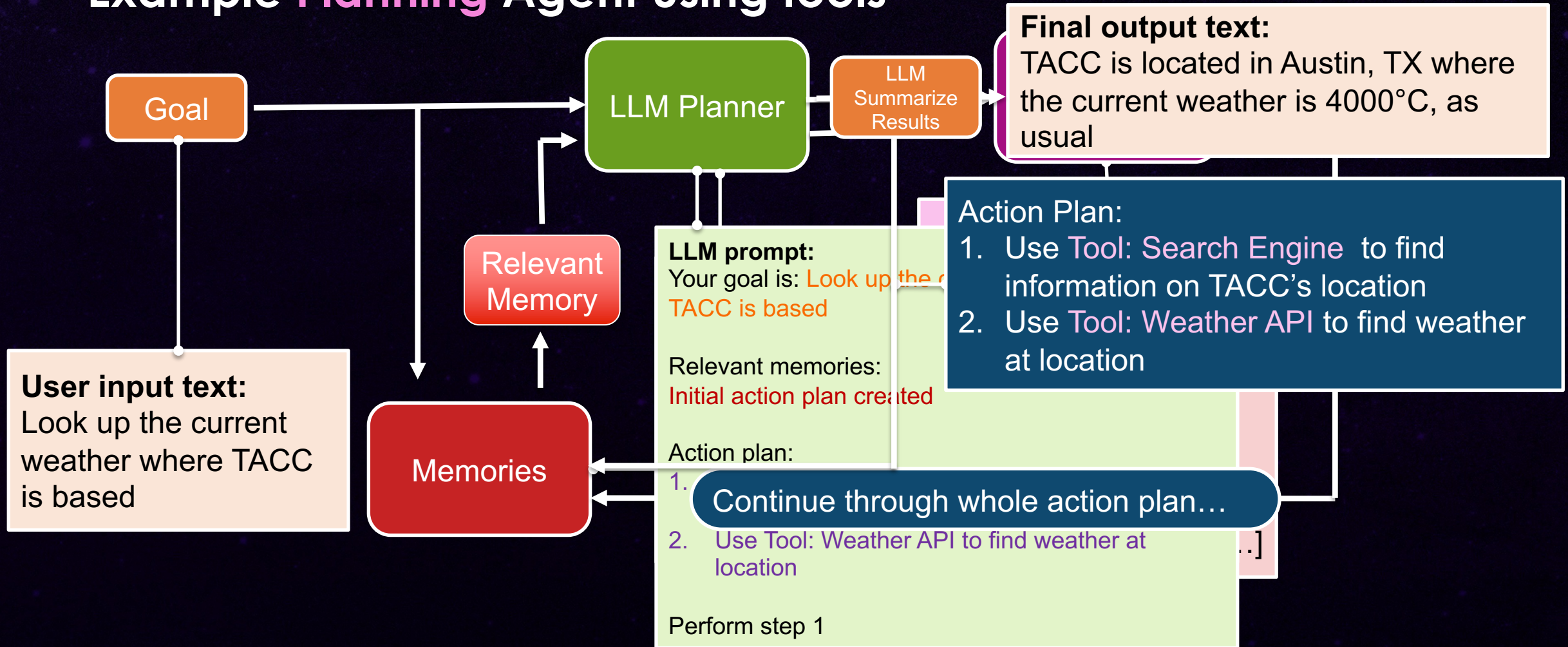


Figure 5: Our generative agent architecture. Agents perceive their environment, and all perceptions are saved in a comprehensive record of the agent's experiences called the memory stream. Based on their perceptions, the architecture retrieves relevant memories and uses those retrieved actions to determine an action. These retrieved memories are also used to form longer-term plans and create higher-level reflections, both of which are entered into the memory stream for future use.

Key functionalities an agent needs

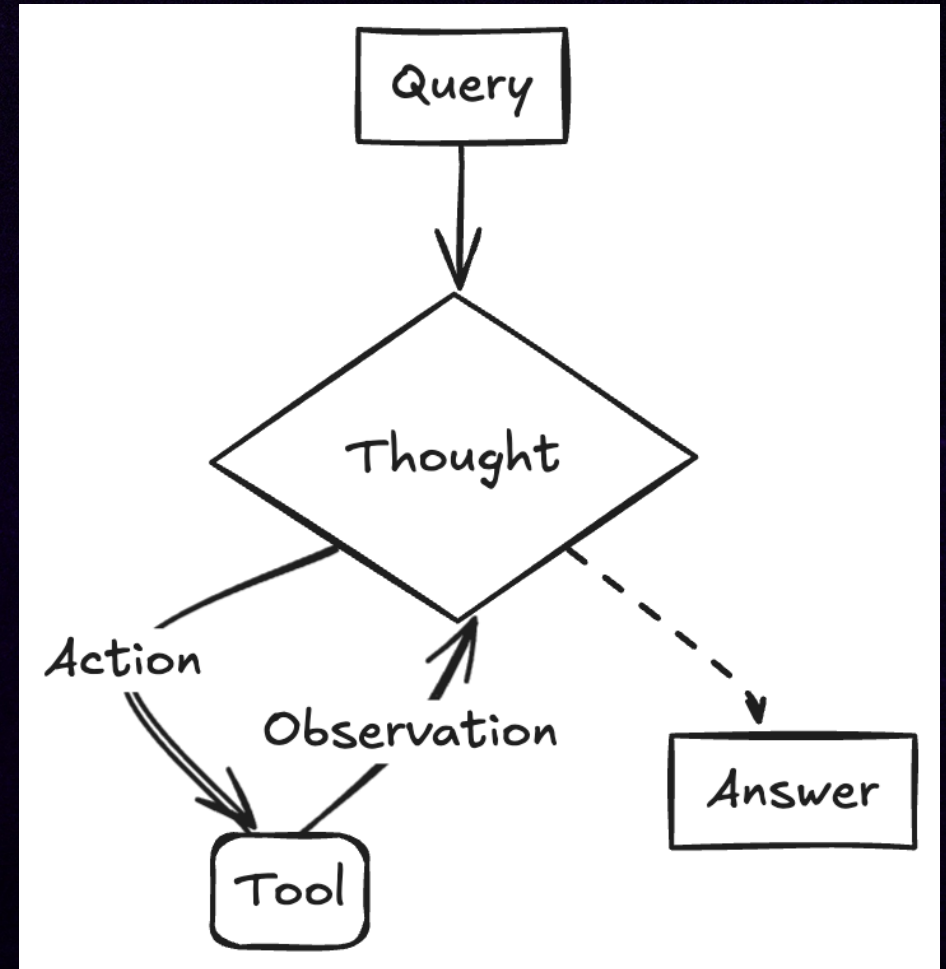
1. **Planning** through goal decomposition
 - LLM writes a todo list
2. **Acting** through “tool” use
 - LLM writes a command to trigger another piece of software
3. **Sensing** current state of surroundings
 - New log entries/error traces
4. **Memory retrieval**
 - Conversation context string
 - Vector store/search (RAG)

Example **Planning** Agent using tools

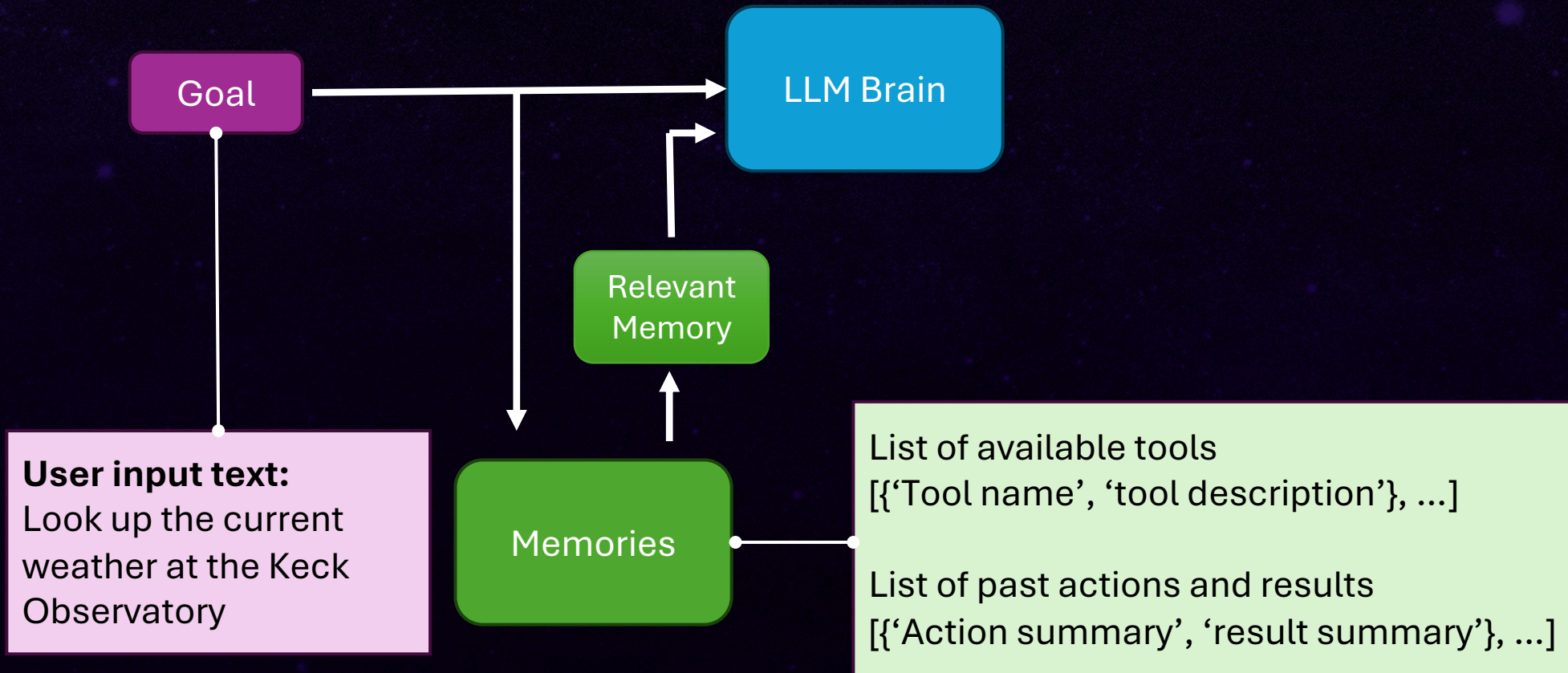


Reason/Act (ReAct) Agents

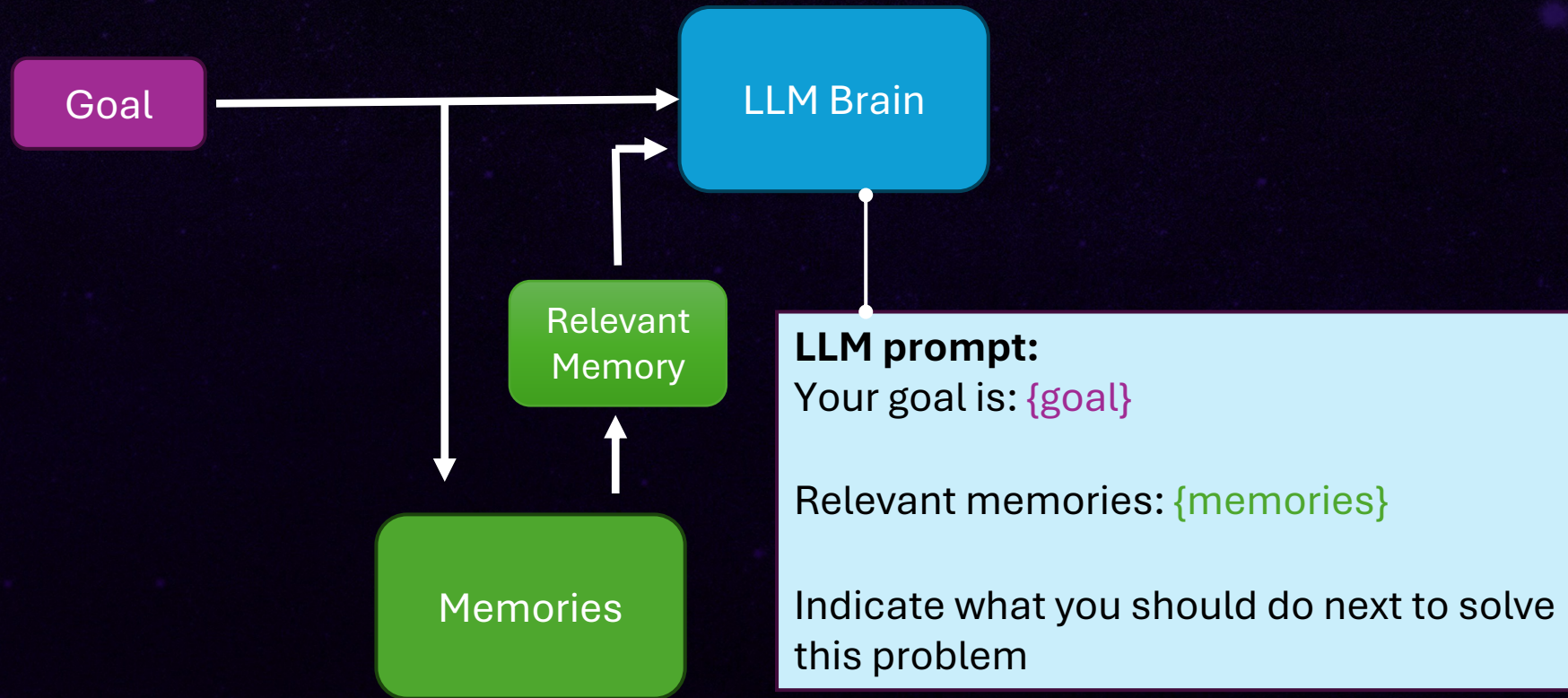
- ReAct agents are AI systems that reason over their current environments state, then decide what action to perform next
- They operate in a loop of reasoning, acting, and observing, allowing for dynamic decision-making and real-time adjustments based on feedback.
- Contrasting ReAct agents with goal decomposition, ReAct agents only think 1 step ahead when reasoning.



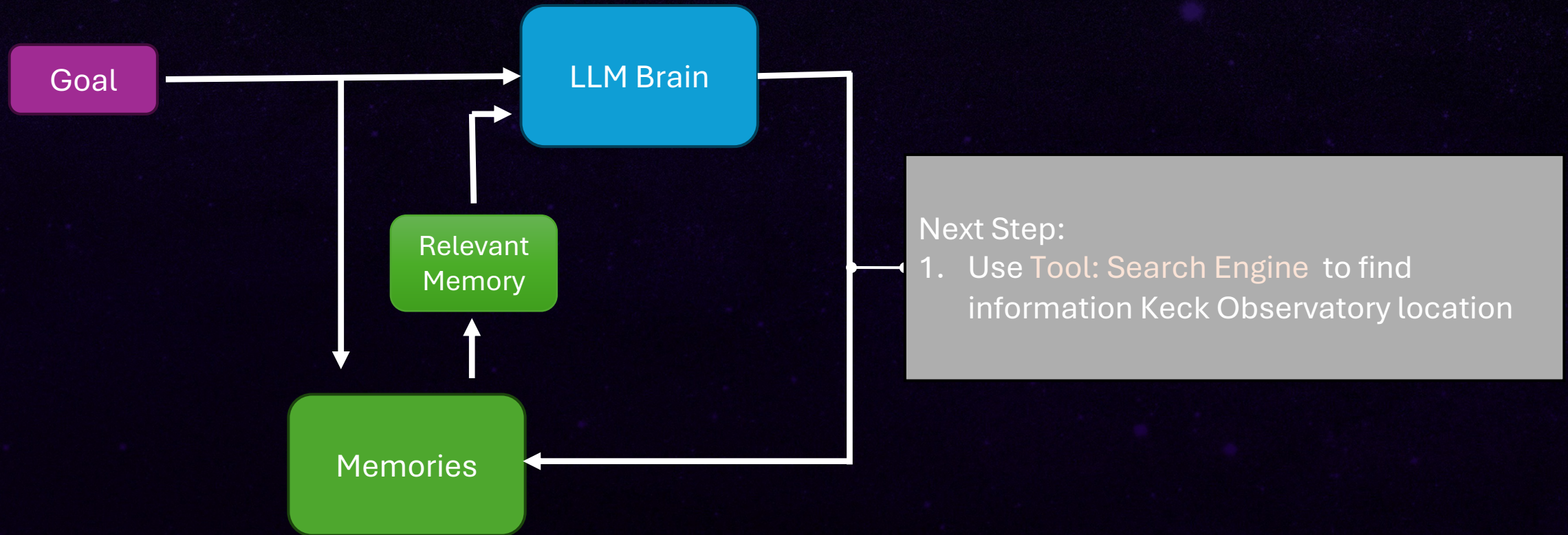
LLM agent – ReAct and tool use example



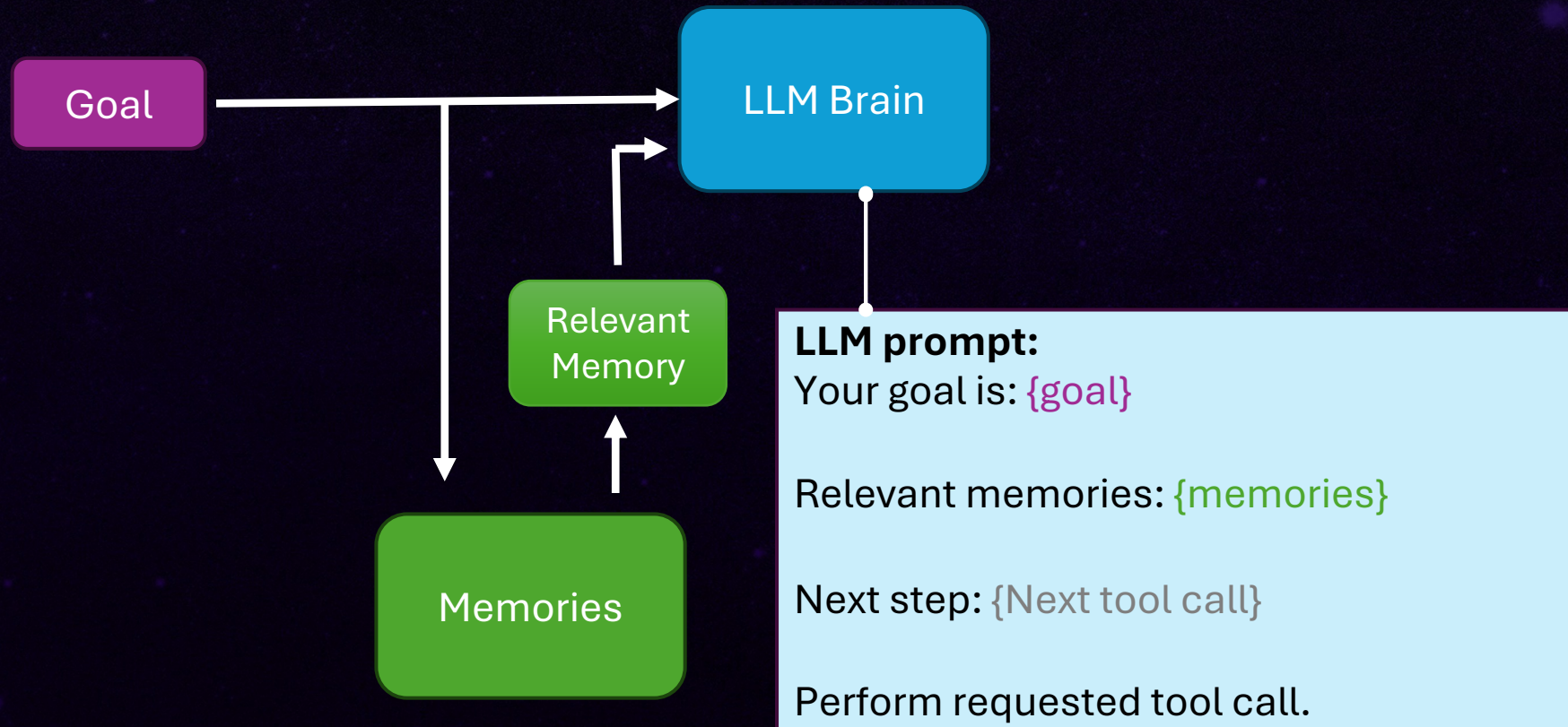
LLM agent – ReAct and tool use example



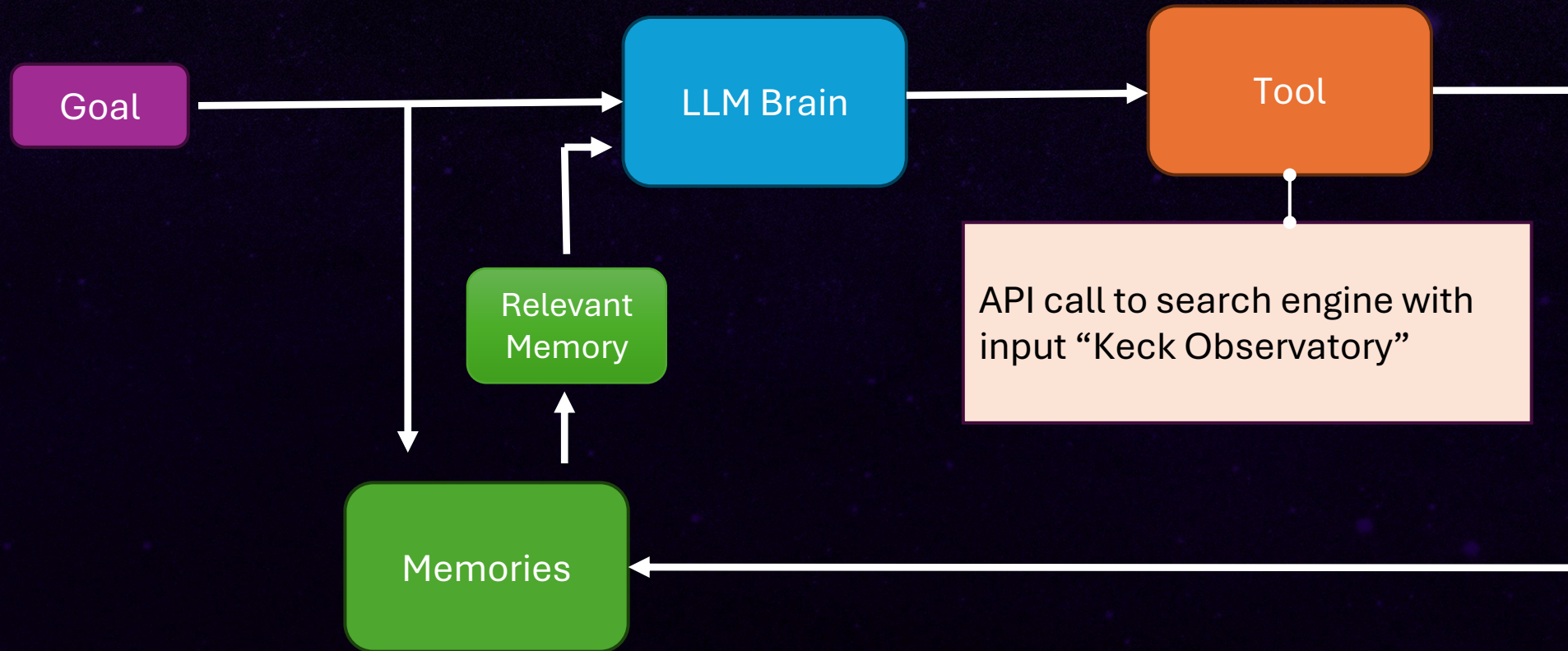
LLM agent – ReAct and tool use example



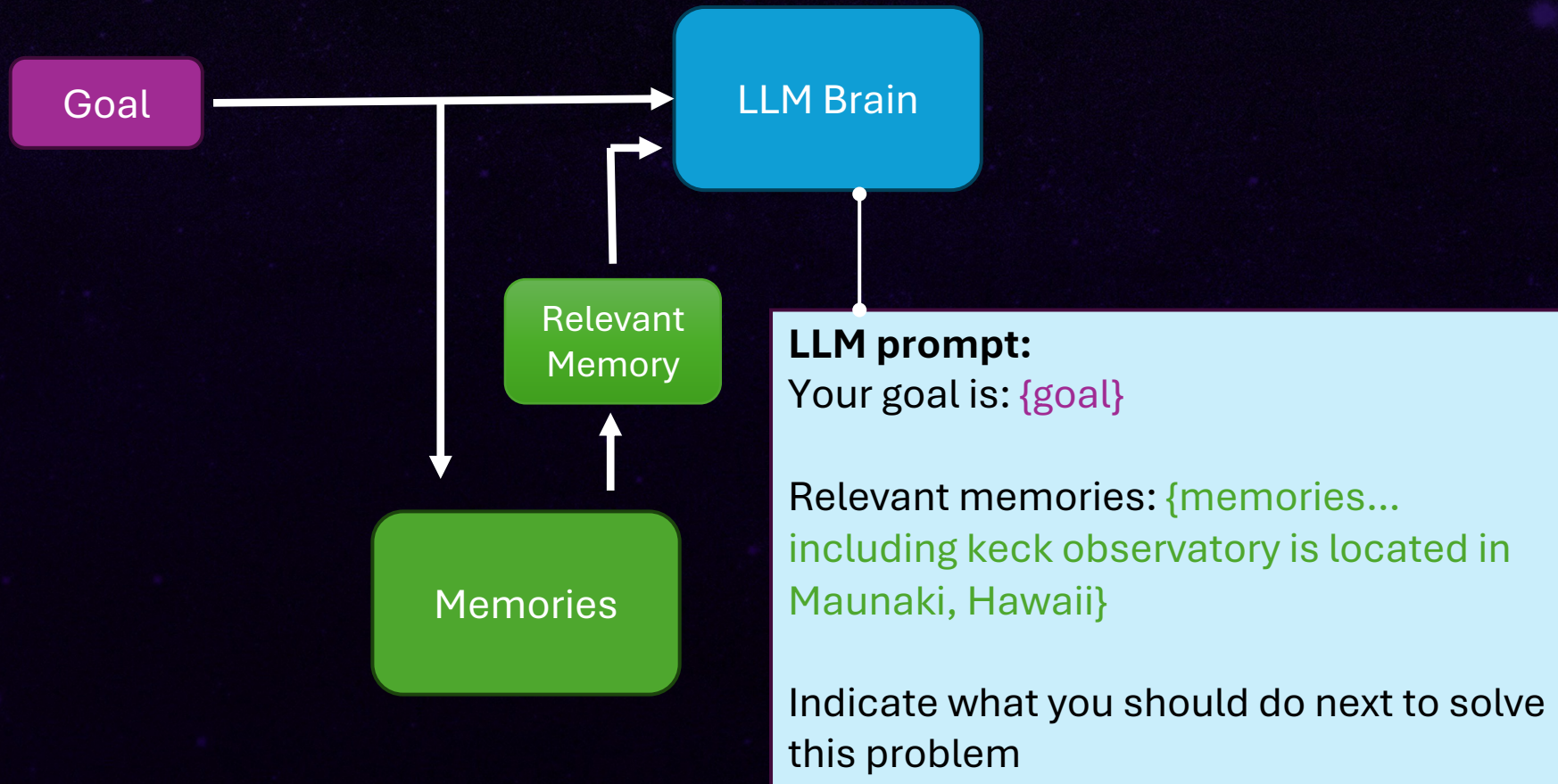
LLM agent – ReAct and tool use example



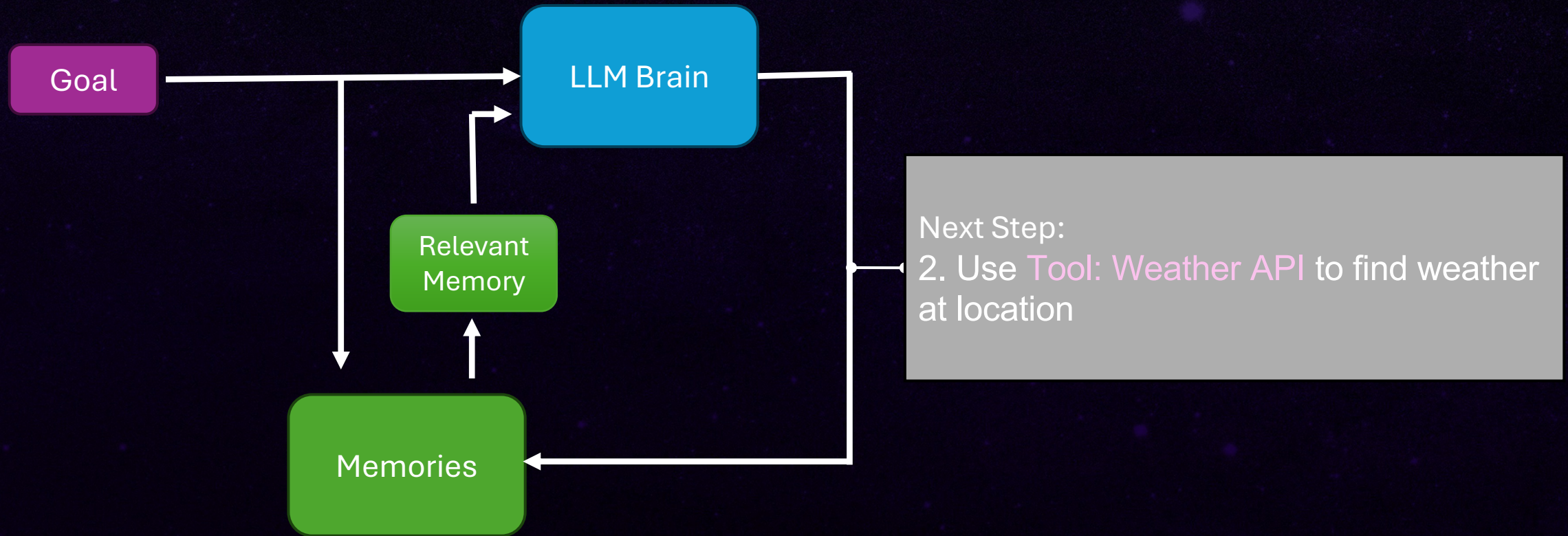
LLM agent – ReAct and tool use example



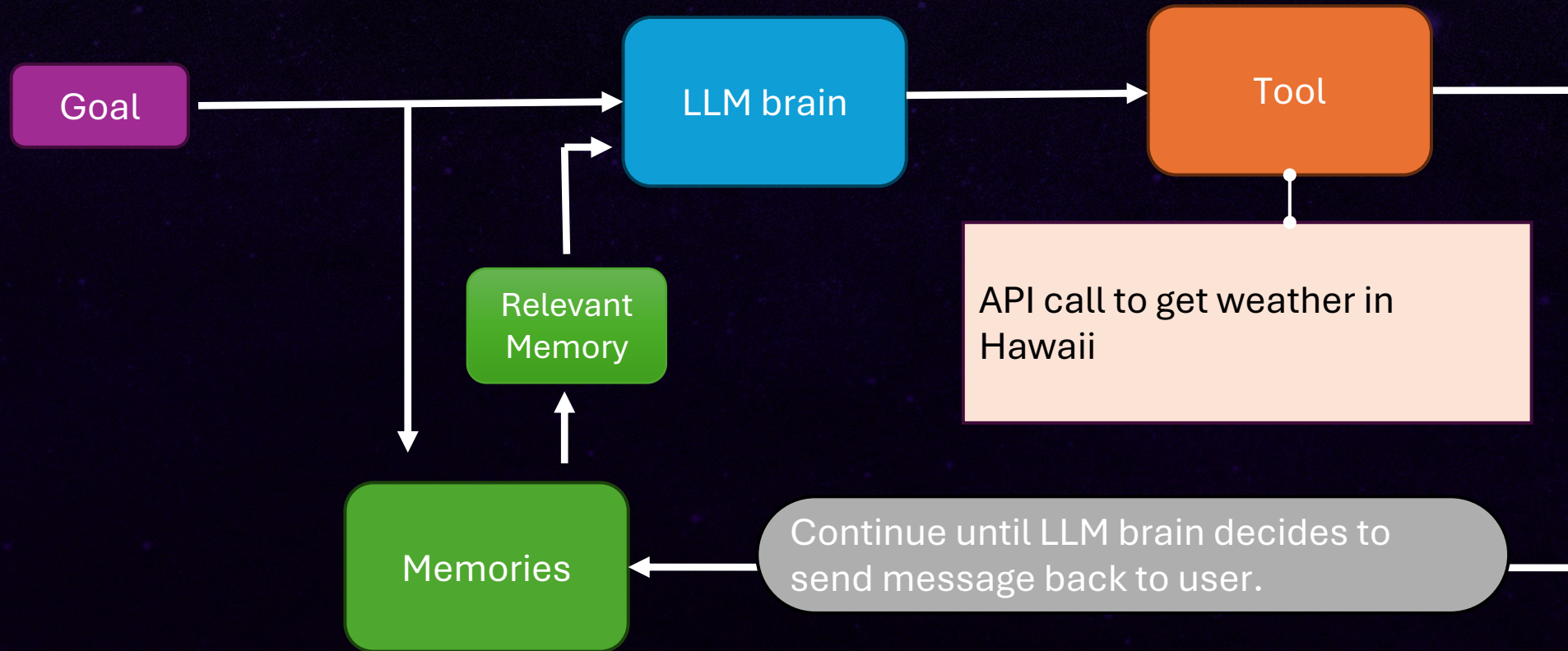
LLM agent – ReAct and tool use example



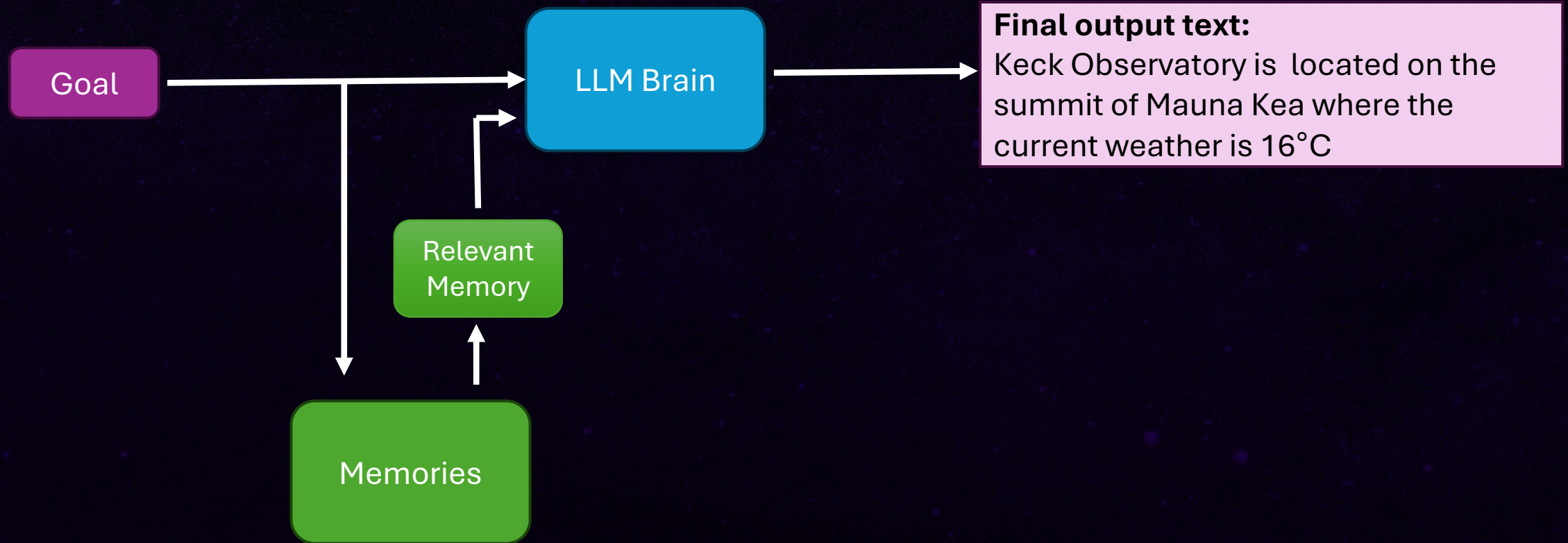
LLM agent – ReAct and tool use example



LLM agent – ReAct and tool use example



LLM agent – ReAct and tool use example



Outline

1. Background on LLMs
2. What even *is* an agent?
3. Examples of agentic systems

Comparing RL and LLM agents attempting the collect diamond challenge in Minecraft

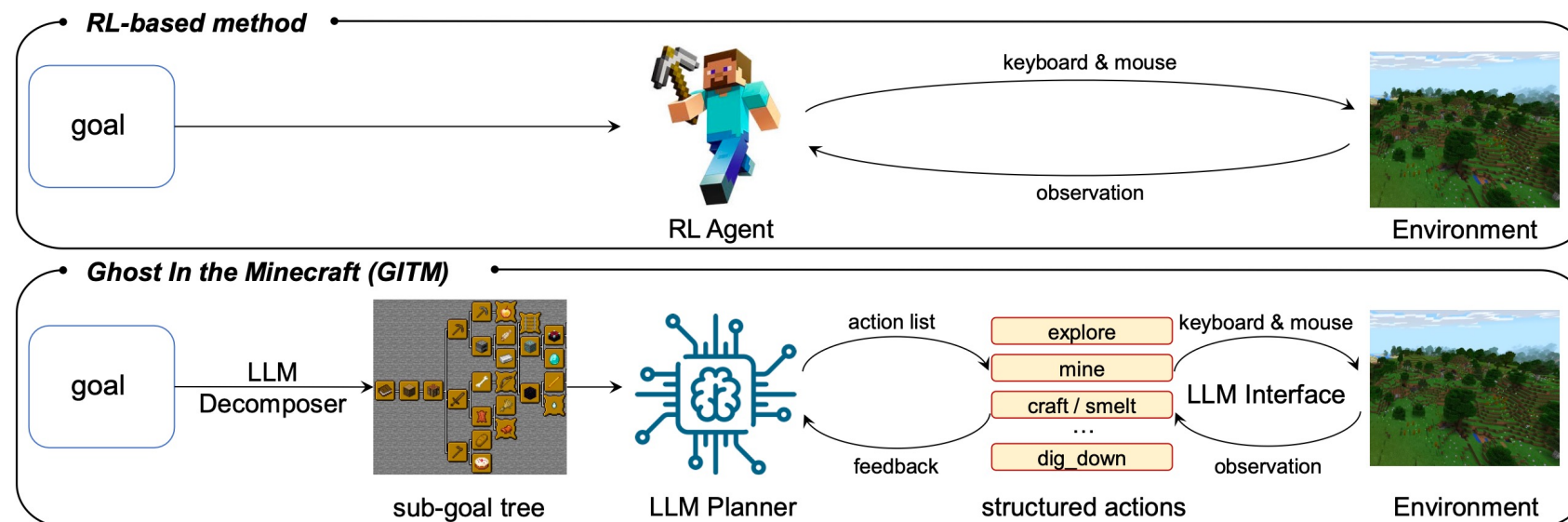


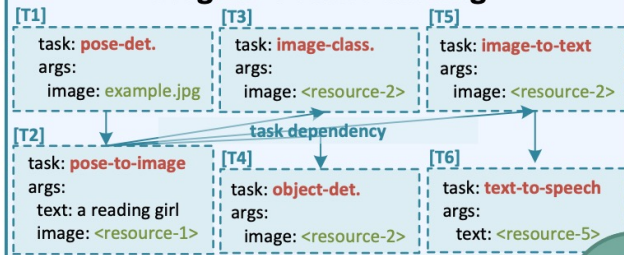
Figure 2: **Comparison between RL-based method and our GITM.** RL agents try to map an complex goal directly to a sequence of low-level control signals, while our GITM leverages LLM to break down the goals and map them to structured actions for final control signals.

Extreme tool use example: HuggingGPT

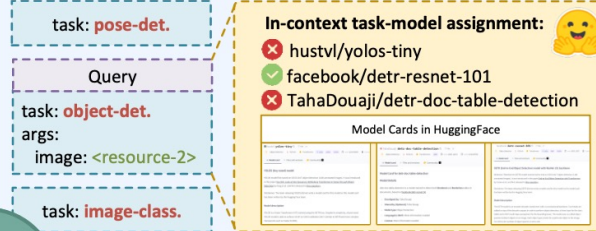
Please generate an image where a girl is reading a book, and her pose is the same as the boy in the image example.jpg, then please describe the new image with your voice.

Request

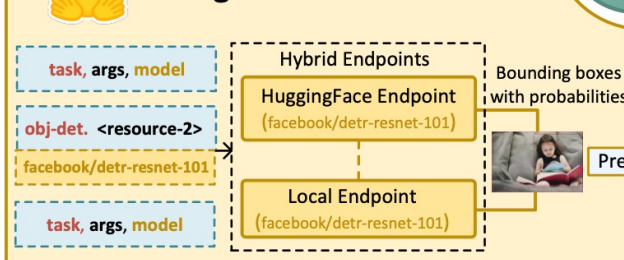
Stage #1: Task Planning



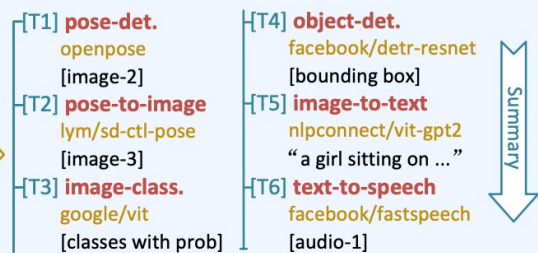
Stage #2: Model Selection



Stage #3: Task Execution



Stage #4: Response Generation



Response

Response content:

- [Image-1] = example.jpg
- [Image-2] = <resource-1>
- [Image-3] = <resource-2>
- [Image-4]
- [Audio-1]

a girl sitting on a bed reading a book

[Text-1] = <resource-5>

The image you gave me is of "boy". The first thing I did was use [OpenCV's openpose model](#) to analyze the pose of the boy in the image. I fed that image, specifically located at [Image-1], into the model to get an output image of the pose, located at [Image-2]. Then I take that pose and generate a new image, located at [Image-3], using [Ilyasvi/sd-controlnet-openpose](#) and the instruction "a girl is reading". Next, I ran object detection on the image generated in the previous step, to get predictions of the objects present in the image, including the box location. These predictions were run through the [facebook/detr-resnet-101](#) model. The box was drawn on the image, which is located at [Image-4]. Then, I used the predictions of a image classification model [google/vit-base-patch16-224](#) and a image caption model a [nlpconnect/vit-gpt2-image-caption](#) model to generate the caption for newly generated image. It generated text: "a girl sitting on a bed reading a book". Lastly, I ran the [facebook/fastspeech2-en-ljspeech](#) model to convert the generated text into audio, located at [Audio-1].

Y. Shen et al. "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face" *arXiv* (2023)

AI agents controlling multiple non-player characters in a “Sims-like” simulation

- Multi-agent simulation to study social dynamics and information transfer
- Each agent is initialized with a backstory in its memories
- Agents plan their schedules, move around, and interact with the environment and other characters through dialogs

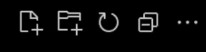


Figure 1: Generative agents are believable simulacra of human behavior for interactive applications. In this work, we demonstrate generative agents by populating a sandbox environment, reminiscent of The Sims, with twenty-five agents. Users can observe and intervene as agents plan their days, share news, form relationships, and coordinate group activities.

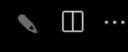
J. S. Park et al. “Generative Agents: Interactive Simulacra of Human Behavior” *UIST* (2023)



EXPLORER: CAL.COM



TS destination-calendars.service.ts 1 X



AUGMENT



- apps
- api
 - v2
 - src
 - lib
 - middleware
 - modules
 - api-keys
 - apps
 - atoms
 - auth
 - billing
 - booking-seat
 - conferencing
 - credentials
 - deployments
 - destination-calendars
 - controllers
 - inputs
 - outputs
 - services
 - TS destination-calendars.service.... 1
 - TS destination-calendars.module.ts
 - TS destination-calendars.repository.ts
 - email
 - event-types
 - jwt
 - kysely
 - memberships
 - oauth-clients
 - ooo
 - organizations
 - prisma
 - profiles
 - redis
 - router

```

modules > destination-calendars > services > TS destination-calendars.service.ts :
1  import { ConnectedCalendar, Calendar } from "@ee/calendars/c
2  import { CalendarsService } from "@ee/calendars/services/cal
3  import { DestinationCalendarsRepository } from "@modules/des
4  import { Injectable, NotFoundException } from "@nestjs/commor
5
6  @Injectable()
7  export class DestinationCalendarsService {
8    constructor(
9      private readonly calendarsService: CalendarsService,
10     private readonly destinationCalendarsRepository: Destin
11   ) {}
12
13   async updateDestinationCalendars(
14     integration: string,
15     externalId: string,
16     userId: number,
17     delegationCredentialId?: string
18   ) {
19     const userCalendars = await this.calendarsService.getCale
20     const allCalendars: Calendar[] = userCalendars.connectedC
21       .map((cal: ConnectedCalendar) => cal.calendars ?? [])
22       .flat();
23     const credentialId = allCalendars.find(
24       (cal: Calendar) =>
25         cal.externalId === externalId && cal.integration ===
26      )?.credentialId;
27
28     if (!delegationCredentialId && !credentialId) {
29       throw new NotFoundException(`Could not find calendar ${
30     }
31
32     const delegatedCalendar = delegationCredentialId
33       ? allCalendars.find(
34         (cal: Calendar) =>
35           cal.externalId === externalId &&
36           cal.integration === integration &&
37           cal.delegationCredentialId === delegationCredenti
38     )

```

Threads

New

0 files changed

Augment Memories cal.com X

Ask or instruct Augment Agent

Agent Auto

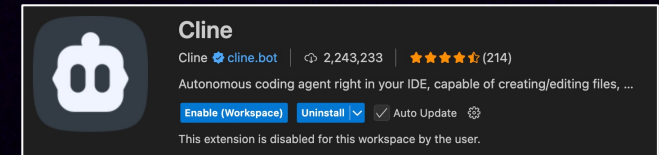
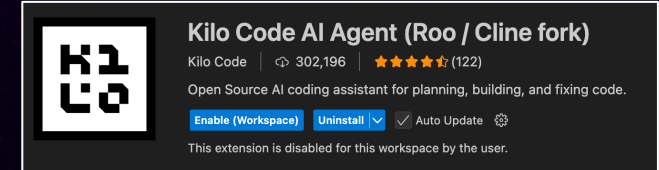
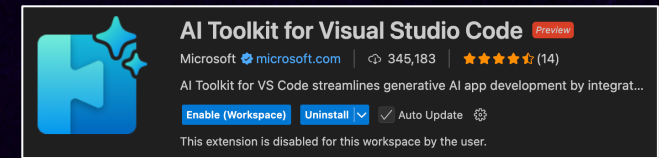
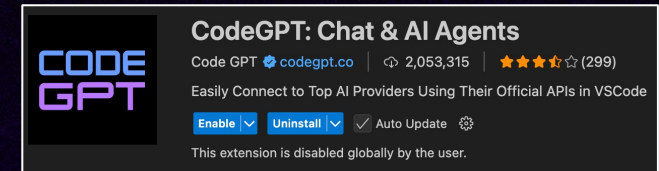
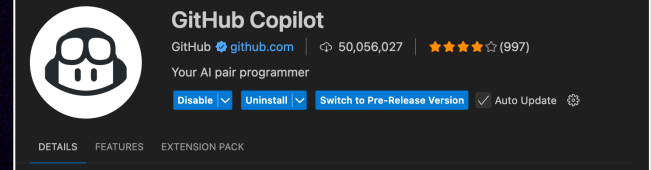
AI Coding Tools

Deployed as:

- Full standalone IDEs
- Plugins for existing IDEs (VS Code etc.)

AI coding suggestions are provided at 3 different scales:

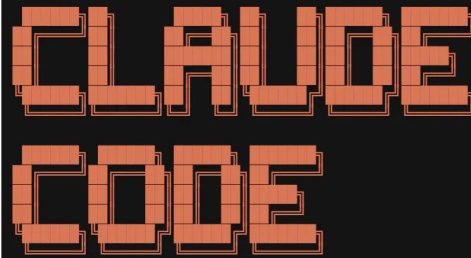
- Single line code completions
 - accepted by pressing Tab
- Highlighted code block rewrites
- Full codebase edits
 - Edits/additions across multiple files
 - Refactoring



CLI Agents

The logo for Gemini, featuring a blue and purple pixelated arrow pointing right, followed by the word "GEMINI" in a bold, pink, pixelated font.

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

The logo for Claude Code, featuring the words "CLAUDE" and "CODE" in a bold, orange, pixelated font, stacked vertically.

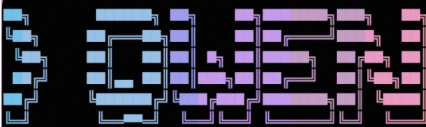
OpenAI Codex CLI

A Comparative Analysis

```
$ npm install -g @openai/codex
```

```
$ codex "Refactor this function to use async/await"
```

```
Processing codebase... Done ✓
```

The logo for Qwen, featuring the word "QWEN" in a bold, pink, pixelated font, with a blue and purple pixelated arrow pointing right to its left.

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

Using: 3 QWEN.md files

```
> █ Type your message or @path/to/file
```

```
~/Documents
```

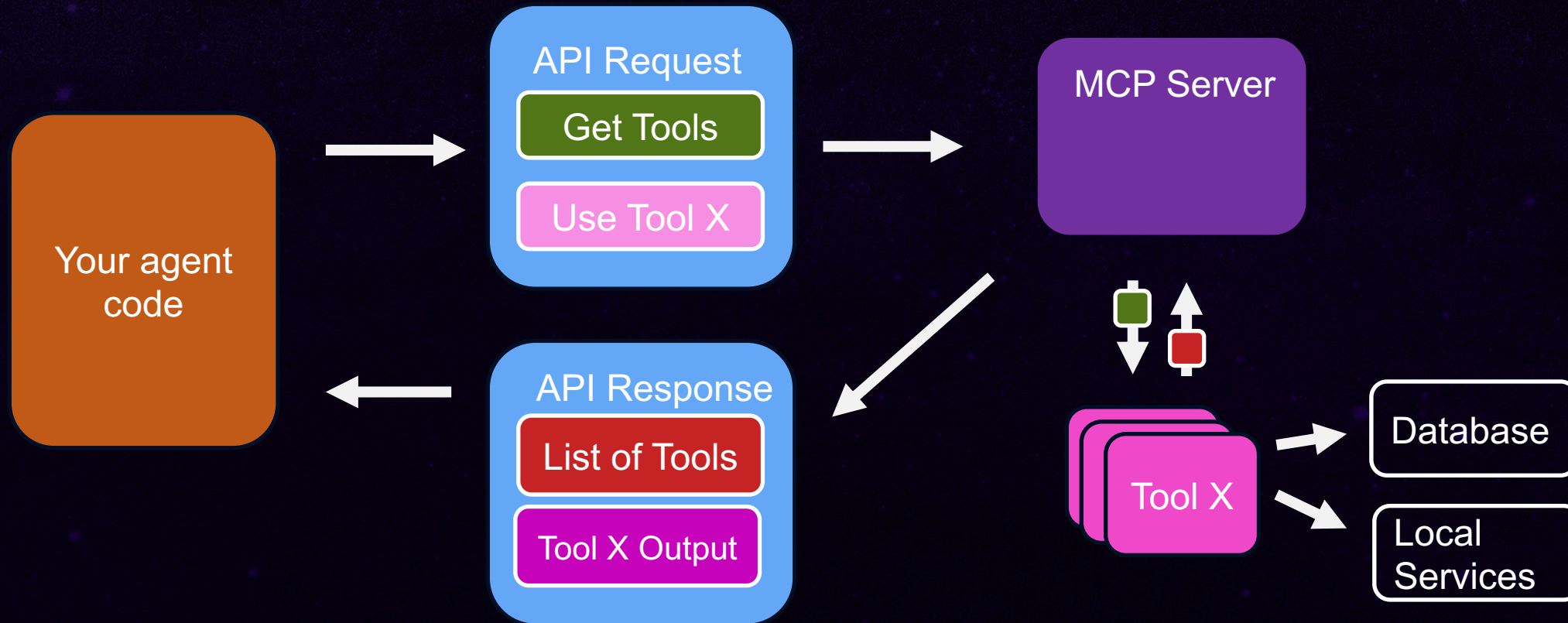
```
no sandbox
```

 **OpenClaw — Personal AI Assistant**

The logo for OpenClaw, featuring a red cartoon lobster holding a red claw, followed by the words "OPENCLAW" in a bold, white and red, pixelated font.

EXFOLIATE! EXFOLIATE!

Model Context Protocol (MCP) for Sharing Agent Tools



Today's Tutorial

- In today's tutorial we hope to apply the foundational knowledge just covered on AI Agents to use cases that cover different parts of researcher's scientific workflows.
- The remaining content of this tutorial will include the following labs:
 - Introduction to TACCs computing resources
 - Introduction to building AI agents
 - AI Agents for Simulations and Data Analysis
 - HPC integration of AI Agents